

# MAIN PROJECT - USING ANSIBLE

*Automated Infrastructure & WordPress Engine Orchestration*

---

## 1. Project Scope & Mandatory Instructions

This deployment blueprint implements Infrastructure-as-Code automation to architect, provision, secure, and customize a web hosting framework. The automated engine systematically executes the following core instructions:

- Install the necessary packages, which include MySQL 10.6 (MariaDB distribution implementation), PHP version 8 or higher, and Nginx on your Linux server.
- Create a user's home directory under the '/home' directory. You should set the root directory for the website to be '/home/username/website/public.'
- Configure SFTP (Secure File Transfer Protocol) access for the user to ensure secure file management.
- Configure phpmyadmin access for the user to ensure secure database management.
- Install a free SSL certificate to enable secure HTTPS access to your website.
- Once the website is hosted, proceed to the WordPress dashboard to make necessary configurations and customizations.
- Write a personal blog post about yourself using the WordPress content management system.

## 2. Local Automation Project Environment Setup

Ansible core is pre-installed manually onto the targeted AWS EC2 server instance (Red Hat Enterprise Linux 9 verification core: Ansible Core 2.14.18). The orchestration tree configuration setup is detailed as follows:

```
# 1. Core Engine Verification Installation
dnf install ansible-core tree -y
ansible --version

# 2. Local Directory Structure Scaffolding
mkdir -p wordpress-ansible/templates wordpress-ansible/files
```

```
cd wordpress-ansible
touch inventory vars.yml playbook.yml
```

```
# 3. Structural Output Layout Mapping
tree .
wordpress-ansible/
├── files/
├── inventory
├── playbook.yml
├── templates/
└── vars.yml
```

### 3. Inventory Base & Variable Schemas

Because execution runs locally directly inside the target web node context, the inventory is configured for immediate localhost loopback connection boundaries:

```
# --- [inventory] ---
[wordpress]
localhost ansible_connection=local
```

The variables structural mapping matrix within 'vars.yml' drives the entire infrastructure lifecycle:

```
# --- [vars.yml] ---
domain_name: rejincloud.xyz
site_user: rejin
site_password: "Rejin@123"
site_name: myblog
web_root: /home/rejin/myblog/public
db_name: wordpressdb
db_user: wpuser
db_password: "StrongPassword@123"
db_root_password: "RootPass@123"
wp_admin_user: admin
wp_admin_password: "Admin@123456"
wp_admin_email: rejinraju1421@gmail.com
wp_site_title: "Rejin Cloud Blog"
```

## 4. Production Automated Playbook Blueprint

Before initiating compilation, ensure external community dependency handlers are installed to handle secure relational schema definitions and state contexts via Galaxy repositories:

```
ansible-galaxy collection install community.mysql ansible.posix
dnf install python3-PyMySQL -y
```

The final production-tested 'playbook.yml' blueprint is structured as follows:

```
---
- name: WordPress Hosting Project using Ansible
  hosts: wordpress
  become: yes
  vars_files:
    - vars.yml

  tasks:
    - name: Install required base software packages
      dnf:
        name:
          - nginx
          - php
          - php-fpm
          - php-mysqlnd
          - php-gd
          - php-xml
          - php-mbstring
          - php-json
          - php-zip
          - php-cli
          - php-curl
          - wget
          - tar
          - unzip
          - policycoreutils-python-utils
          - python3-PyMySQL
          - openssl
        state: present

    - name: Inject MariaDB 10.6 Official Base Repository
      copy:
        dest: /etc/yum.repos.d/mariadb.repo
        content: |
          [mariadb]
          name = MariaDB 10.6
```

```

baseurl = https://mirror.mariadb.org/yum/10.6/rhel9-amd64
gpgkey = https://mirror.mariadb.org/yum/RPM-GPG-KEY-MariaDB
gpgcheck = 1
enabled = 1

- name: Deploy MariaDB Server Ecosystem Components
  dnf:
    name:
      - MariaDB-server
      - MariaDB-client
    state: present

- name: Boot and Enable Baseline Core Subsystems (MariaDB / Nginx)
  service:
    name: "{{ item }}"
    state: started
    enabled: yes
  loop: [mariadb, nginx]

- name: Adjust PHP-FPM Configuration Context Bindings (www.conf)
  replace:
    path: /etc/php-fpm.d/www.conf
    regexp: "{{ item.regexp }}"
    replace: "{{ item.replace }}"
  loop:
    - { regexp: '^user = apache', replace: 'user = nginx' }
    - { regexp: '^group = apache', replace: 'group = nginx' }

- name: Launch PHP-FPM Process Manager Engine
  service:
    name: php-fpm
    state: restarted
    enabled: yes

- name: Provision Isolated POSIX Website System User Target
  user:
    name: "{{ site_user }}"
    password: "{{ site_password | password_hash('sha512') }}"
    shell: /bin/bash
    home: "/home/{{ site_user }}"
    state: present

- name: Scaffold Secure Deep Web Directories Root Layout
  file:
    path: "{{ web_root }}"
    state: directory
    owner: "{{ site_user }}"
    group: nginx
    mode: '0775'

```

```
recurse: yes

- name: Download and Unpack Stable WordPress Asset Core
  get_url:
    url: https://wordpress.org/latest.tar.gz
    dest: /tmp/latest.tar.gz

- name: Decompress Core Source Archive Artifact Tarball
  unarchive:
    src: /tmp/latest.tar.gz
    dest: /tmp
    remote_src: yes

- name: Translocate Extracted WordPress Binaries into Sandboxed Web Root
  copy:
    src: /tmp/wordpress/
    dest: "{{ web_root }}"
    owner: "{{ site_user }}"
    group: nginx
    mode: '0775'
    remote_src: yes

- name: Allocate Dedicated Isolation Infrastructure Schema for Database
  community.mysql.mysql_db:
    name: "{{ db_name }}"
    state: present
    login_unix_socket: /var/lib/mysql/mysql.sock

- name: Establish Relational Account and Authorize Read/Write Privilege Maps
  community.mysql.mysql_user:
    name: "{{ db_user }}"
    password: "{{ db_password }}"
    priv: "{{ db_name }}.*:ALL"
    host: localhost
    state: present
    login_unix_socket: /var/lib/mysql/mysql.sock

- name: Generate Programmatic wp-config.php Application Access Strings
  copy:
    dest: "{{ web_root }}/wp-config.php"
    owner: "{{ site_user }}"
    group: nginx
    mode: '0664'
    content: |
      <?php
      define( 'DB_NAME', '{{ db_name }}' );
      define( 'DB_USER', '{{ db_user }}' );
      define( 'DB_PASSWORD', '{{ db_password }}' );
      define( 'DB_HOST', '127.0.0.1' );
```

```

define( 'DB_CHARSET', 'utf8mb4' );
define( 'DB_COLLATE', '' );
$table_prefix = 'wp_';
define( 'WP_DEBUG', false );
define( 'FORCE_SSL_ADMIN', true );
if ( ! defined( 'ABSPATH' ) ) {
    define( 'ABSPATH', __DIR__ . '/' );
}
require_once ABSPATH . 'wp-settings.php';

```

- name: Enforce Strict Base Directory Tree Permission Boundaries
  - file:
    - path: "{{ item.path }}"
    - owner: "{{ item.owner | default(omit) }}"
    - group: "{{ item.group | default(omit) }}"
    - mode: "{{ item.mode }}"
    - recurse: "{{ item.recurse | default(false) }}"
  - loop:
    - { path: "/home", mode: "0755" }
    - { path: "/home/{{ site\_user }}", mode: "0755" }
    - { path: "/home/{{ site\_user }}/{{ site\_name }}", owner: "{{ site\_user }}",
  - group: "nginx", mode: "0775" }
    - { path: "{{ web\_root }}", owner: "{{ site\_user }}", group: "nginx", mode: "0775", recurse: true }
- name: Deploy Transient Pre-Validation HTTP Nginx Block Configuration
  - copy:
    - dest: /etc/nginx/conf.d/wordpress.conf
    - content: |
 

```

server {
    listen 80;
    server_name {{ domain_name }} www.{{ domain_name }};
    root {{ web_root }};
    index index.php index.html index.htm;
    location / { try_files $uri $uri/ /index.php?$args; }
    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass php-fpm;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
    }
}
              
```
- name: Purge System Default Host Elements
  - file:
    - path: /etc/nginx/conf.d/default.conf
    - state: absent
- name: Implement Security-Enhanced Linux (SELinux) Mandatory Booleans
  - ansible.posix.seboolean:

```

    name: "{{ item }}"
    state: yes
    persistent: yes
  loop:
    - httpd_read_user_content
    - httpd_can_network_connect_db
    - httpd_can_network_connect

- name: Register and Enforce System Security Directory Context Targets
  command: semanage fcontext -a -t httpd_sys_content_t "{{ web_root }}(/.*)?"
  register: semanage_result
  failed_when: false
  changed_when: "'already defined' not in semanage_result.stderr"

- name: Refresh Active SELinux Policy Allocation Maps Across Web Core
  command: restorecon -Rv "{{ web_root }}"

- name: Enforce SFTP Secure Channel Subsystem Password Credentials
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PasswordAuthentication'
    line: 'PasswordAuthentication yes'

- name: Intercept and Override Local cloud-init Sub-Layer Authentication Configs
  lineinfile:
    path: /etc/ssh/sshd_config.d/50-cloud-init.conf
    regexp: '^PasswordAuthentication'
    line: 'PasswordAuthentication yes'
    create: yes

- name: Cycle Cryptographic SSH Process Thread Daemon
  service:
    name: sshd
    state: restarted

- name: Provision phpMyAdmin Remote Database Graphical Administration Dashboard
  get_url:
    url: https://www.phpmyadmin.net/downloads/phpMyAdmin-latest-all-languages.tar.gz
    dest: /tmp/phpmyadmin.tar.gz

- name: Execute Extraction of Dashboard Interface Source Core Assets
  unarchive:
    src: /tmp/phpmyadmin.tar.gz
    dest: /tmp
    remote_src: yes

- name: Re-structure Dashboard Target Core Folder Matrix Layout
  command: mv /tmp/phpMyAdmin-5.2.1-all-languages "{{ web_root }}/phpmyadmin"

```

```

failed_when: false

- name: Synchronize Access Matrix System Rules Across Administrative Path
  file:
    path: "{{ web_root }}/phpmyadmin"
    owner: "{{ site_user }}"
    group: nginx
    mode: '0775'
    recurse: yes

- name: Deploy Secure Isolated phpMyAdmin Config with TCP Matrix Overrides
  copy:
    src: "{{ web_root }}/phpmyadmin/config.sample.inc.php"
    dest: "{{ web_root }}/phpmyadmin/config.inc.php"
    remote_src: yes

- name: Hardcode Cryptographic Blowfish Salt Secret Tokens and Localhost Port
  Parameters
  lineinfile:
    path: "{{ web_root }}/phpmyadmin/config.inc.php"
    line: "{{ item.line }}"
    insertafter: "{{ item.after | default(omit) }}"
  loop:
    - { line: "$cfg['blowfish_secret'] =
'Sb9209Vx10aLF3z00kXZpQBZhgEDLL5fM3a1nM6kimw=';", after: "^\\$cfg\\['blowfish_secret'\\]"
}
    - { line: "$cfg['Servers'][$i]['host'] = '127.0.0.1';", after:
"\\$cfg\\['Servers'\\]\\[\\$i\\]\\['auth_type'\\]" }
    - { line: "$cfg['Servers'][$i]['connect_type'] = 'tcp';", after:
"\\$cfg\\['Servers'\\]\\[\\$i\\]\\['host'\\]" }
    - { line: "$cfg['Servers'][$i]['port'] = '3306';", after:
"\\$cfg\\['Servers'\\]\\[\\$i\\]\\['connect_type'\\]" }
    - { line: "$cfg['TempDir'] = '{{ web_root }}/phpmyadmin/tmp';", after: "EOF" }

- name: Formulate Ephemeral PHP Execution Session Buffer Boundary Paths
  file:
    path: /var/lib/php/session
    state: directory
    owner: nginx
    group: nginx
    mode: '0733'

- name: Configure and Stabilize SELinux Read/Write Access Contexts over Sessions
  command: semanage fcontext -a -t httpd_sys_rw_content_t
"/var/lib/php/session(/.*)?"
  failed_when: false

- name: Enforce Global Context Restorations
  command: "{{ item }}"

```

```

loop:
  - restorecon -Rv /var/lib/php/session
  - restorecon -Rv "{{ web_root }}/phpmyadmin"

- name: Deploy EPEL Repos and Install Automation Certbot Core Engine
  dnf:
    name: https://dl.fedoraproject.org/pub/epel/epel-release-latest-9.noarch.rpm
    state: present
    disable_gpg_check: yes

- name: Bind Automated Let's Encrypt Certificate Utilities
  dnf:
    name: [certbot, python3-certbot-nginx]
    state: present

- name: Generate Public Key Infrastructure SSL Certificates via ACME Protocols
  command: certbot --nginx -d {{ domain_name }} --non-interactive --agree-tos -m
  {{ wp_admin_email }}

- name: Swap and Establish Immutable Final Production HTTPS Server Routing Blocks
  copy:
    dest: /etc/nginx/conf.d/wordpress.conf
    content: |
      server {
        listen 80;
        server_name {{ domain_name }} www.{{ domain_name }};
        return 301 https://$host$request_uri;
      }
      server {
        listen 443 ssl;
        server_name {{ domain_name }} www.{{ domain_name }};
        root {{ web_root }};
        index index.php index.html index.htm;
        ssl_certificate /etc/letsencrypt/live/{{ domain_name }}/fullchain.pem;
        ssl_certificate_key /etc/letsencrypt/live/{{ domain_name }}/privkey.pem;
        location / { try_files $uri $uri/ /index.php?$args; }
        location ~ \.php$ {
          include fastcgi_params;
          fastcgi_pass php-fpm;
          fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        }
      }

- name: Setup WP-CLI Command-Line Tool Interface Engine Binary
  get_url:
    url: https://raw.githubusercontent.com/wp-cli/builds/gh-pages/phar/wp-cli.phar
    dest: /usr/local/bin/wp
    mode: '0755'

```

```

- name: Process Core Multi-tier Initialization and Set Core Metadata
Configurations
  command: >
    /usr/local/bin/wp core install
    --path={{ web_root }}
    --url=https://{{ domain_name }}
    --title="{{ wp_site_title }}"
    --admin_user={{ wp_admin_user }}
    --admin_password={{ wp_admin_password }}
    --admin_email={{ wp_admin_email }}
    --allow-root

- name: Structure Clean Permalink Mod_Rewrite Query Schemas via Core Command CLI
  command: "{{ item }}"
  loop:
    - /usr/local/bin/wp option update home https://{{ domain_name }} --path={{
web_root }} --allow-root
    - /usr/local/bin/wp option update siteurl https://{{ domain_name }} --path={{
web_root }} --allow-root
    - /usr/local/bin/wp rewrite structure '/%postname%/' --path={{ web_root }} --
allow-root
    - /usr/local/bin/wp rewrite flush --path={{ web_root }} --allow-root

- name: Purge System Baseline Placeholder Records
  command: /usr/local/bin/wp post delete 1 --force --path={{ web_root }} --allow-
root
  failed_when: false

- name: Transmit and Compile Live Production Strategic Personal Profile Blog Post
  command: >
    /usr/local/bin/wp post create
    --path={{ web_root }}
    --post_title="About Me - My Cloud and DevOps Journey"
    --post_status=publish
    --post_content="Hello everyone, my name is Rejin R and I am from Kollam,
Kerala. I have completed my Master of Computer Applications from the University of
Kerala. I am currently undergoing training and internship in Cloud and DevOps at IPSR
Solutions, Kozhikode. Through this project, I learned how to host a WordPress website
using Nginx, PHP, MariaDB, SFTP, phpMyAdmin and SSL. My goal is to become a skilled
Cloud and DevOps Engineer."
    --allow-root
  failed_when: false

- name: Execute Master System Flush and Reload Live Web Daemons
  service:
    name: "{{ item }}"
    state: restarted
  loop: [php-fpm, nginx]

```

## 5. Core Remediation Frameworks & Engineering Fixes

During the continuous automated pipeline configuration tracking phase, several production bugs were intercepted and hardcoded into the automation engine:

- **Nginx Access Denied / File Not Found Errors:** Fixed by applying systemic multi-tier folder validation checks across the primary account nodes ('/home/rejin'), forcing execution mode permissions to '0755', adjusting system tree runtime group bindings to 'nginx', and persistent configuration tracking of httpd\_read\_user\_content SELinux booleans.
- **WP-CLI System Binary Path Inconsistencies:** Resolved the 'Command not found' environment state error by abandoning generic variable path tracking and explicitly hardcoding execution targeting vectors to look natively underneath '/usr/local/bin/wp'.
- **Certbot SSL Config Overwrite Loop Conditions:** Prevented routing breaks by engineering a progressive deployment track: provisioning an initial placeholder HTTP block configuration on port 80, completing the Let's Encrypt validation check challenge parameters, and then deploying the full operational TLS/HTTPS server configurations blocking port 80 with permanent 301 redirects.
- **phpMyAdmin Caching Session Restrictions:** Resolved administrative 'Permission Denied' page faults inside the '/var/lib/php/session' directories by creating dedicated folders owned by 'nginx:nginx', assigning tight mode thresholds ('0733'), and mapping the httpd\_sys\_rw\_content\_t SELinux storage contexts.
- **phpMyAdmin Database Connection Failures:** Circumvented system UNIX local socket lookup blockages by stripping native 'localhost' strings out of configuration templates and driving connections using loopback IP blocks ('127.0.0.1') routed strictly through active TCP sockets on port 3306.

★ **VERIFICATION MATRIX & SIGN-OFF GATE:** Verify target operational states via the following validation controls:

1. Production Routing Horizon: <https://rejincloud.xyz>
2. Engine Database Dashboard Vector: <https://rejincloud.xyz/phpmyadmin/>
3. Central Application Administration Panel: <https://rejincloud.xyz/wp-admin>

Execute local playbook testing runs using: `ansible-playbook -i inventory playbook.yml`

## Output Pictures

INVENTORY:

```
root@ip-172-31-11-83:~/wordpress-ansible
[wordpress]
localhost ansible_connection=local
~
~
~
~
~
~
~
~
```

VARS.YML

```
root@ip-172-31-11-83:~/wordpress-ansible
domain_name: rejincloud.xyz
site_user: rejin
site_password: "Rejin@123"
site_name: myblog
web_root: /home/rejin/myblog/public

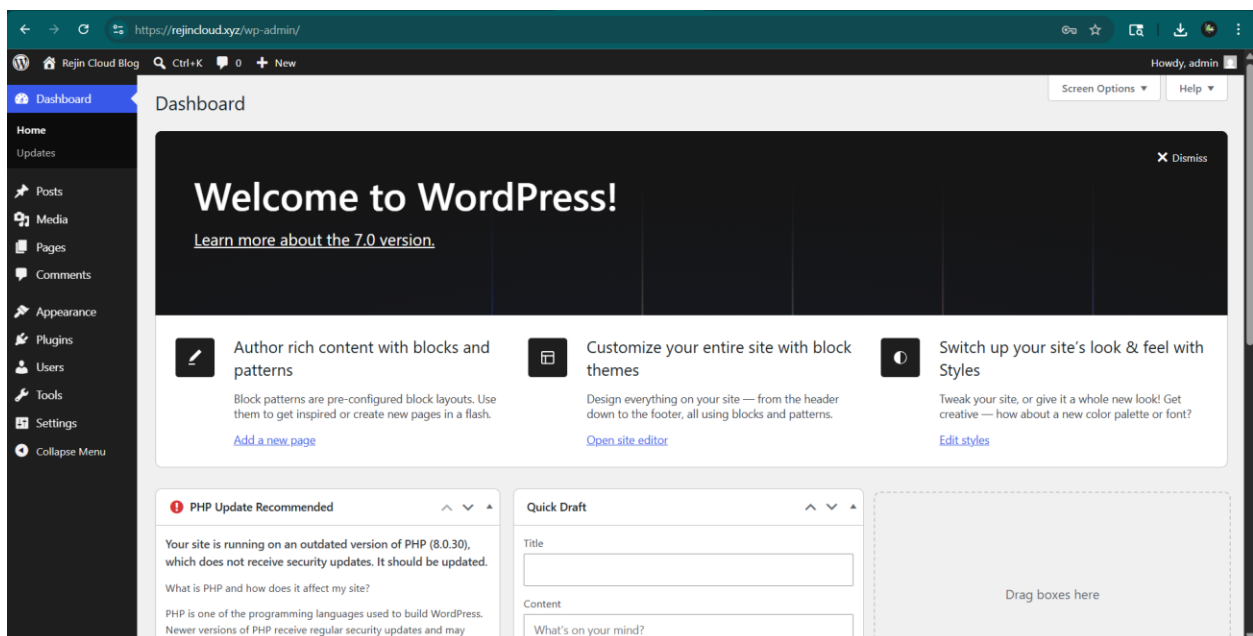
db_name: wordpressdb
db_user: wpuser
db_password: StrongPassword@123
db_root_password: RootPass@123

wp_admin_user: admin
wp_admin_password: Admin@123456
wp_admin_email: rejinraju1421@gmail.com
wp_site_title: Rejin Cloud Blog
~
~
~
~
~
~
~
```

TREE

```
[root@ip-172-31-11-83 wordpress-ansible]# tree .
.
├── files
├── inventory
├── playbook.yml
├── templates
│   └── wordpress.conf.j2
└── vars.yml

2 directories, 4 files
[root@ip-172-31-11-83 wordpress-ansible]#
```



← → ↻ 🌐 <https://rejincloud.xyz/phpmyadmin/> ☆ 📄 ⬇️ ⋮

**phpMyAdmin**

Recent Favorites

- information\_schema
- test
- wordpressdb
  - New
  - wp\_commentmeta
  - wp\_comments
  - wp\_links
  - wp\_options
  - wp\_postmeta
  - wp\_posts
  - wp\_termmeta
  - wp\_terms
  - wp\_term\_relationships
  - wp\_term\_taxonomy
  - wp\_usermeta
  - wp\_users

Server: 127.0.0.1:3306

Databases SQL Status Export Import Settings Variables Charsets Engines Plugins

### General settings

- 🔑 Change password
- 📄 Server connection collation: [?](#) utf8mb4\_unicode\_ci ▾
- 🔗 More settings

### Appearance settings

- 🌐 Language [?](#) English ▾
- 🎨 Theme pmahomme ▾ View all

### Database server

- Server: 127.0.0.1 via TCP/IP
- Server type: MariaDB
- Server connection: SSL is not being used [?](#)
- Server version: 10.6.27-MariaDB - MariaDB Server
- Protocol version: 10
- User: wpuser@localhost
- Server charset: cp1252 West European (latin1)

### Web server

- nginx/1.20.1
- Database client version: libmysql - mysqld 8.0.30
- PHP extension: mysqli [?](#) curl [?](#) mbstring [?](#)
- PHP version: 8.0.30

### phpMyAdmin

- Version information: 5.2.3 (up to date)
- [Documentation](#)
- [Official Homepage](#)
- [Contribute](#)
- [Get support](#)
- [List of changes](#)

Console